



ICS3U Resource Book

Revised August 2013

11.1 Everyday Recursion

We begin our study of recursion by examining situations from everyday life that illustrate recursion.

If you wanted to draw a family tree showing an individual and all of that person's direct descendants, you would probably start by writing down the name of the person. Under his or her name, you could then draw lines to the name of each child of the person. If any of these children had children, you could then draw lines to the names of their children. This process could be continued, drawing lines to the names of the children's children, and so on until you reached a point (for each descendant) at which there were no further descendants.

We can describe this process easily and concisely if we use recursion.

Example 1

To draw a family tree showing a person and all of the person's direct descendants, we can use the following algorithm.

```
To DRAW THE FAMILY TREE of a person
  write the name of the person
  if the person had children
    for each child
      draw a line from the person and
      DRAW THE FAMILY TREE of the child
```

The example illustrates two features typical of recursive algorithms.

1. The simplest case of the problem can be solved directly. In the example, a childless person is the simplest case; after writing the name of such a person, that person's family tree is complete.
2. All other cases of the problem are solved by doing a bit of work in the direction of a solution and then solving one or more sub-problems of the original problem. In the example, after writing the name of a person with one or more children, we must draw the family tree of each of these children.

This type of analysis can be applied to many activities. We illustrate a fairly silly one in the next example.

Example 2

Let us consider the structure of an onion without its brown papery covering. Suppose that we want to describe the process of finding the centre of the onion. If we peel away the outermost thick, fleshy layer, we are either at the core or we are left with what looks like a smaller version of the original onion. Thus, finding the centre of an onion could be described by the following recursive procedure.

```
To FIND THE CENTRE OF THE ONION
  if you are not at the core
    remove one layer
  FIND THE CENTRE OF THE ONION
```

The simplest case for finding the centre of the onion is finding that we are at the core. In the more complex case, we consider an onion as being composed of a fleshy layer that surrounds a smaller onion or the core. By removing one layer, we get a little closer to the simple case.

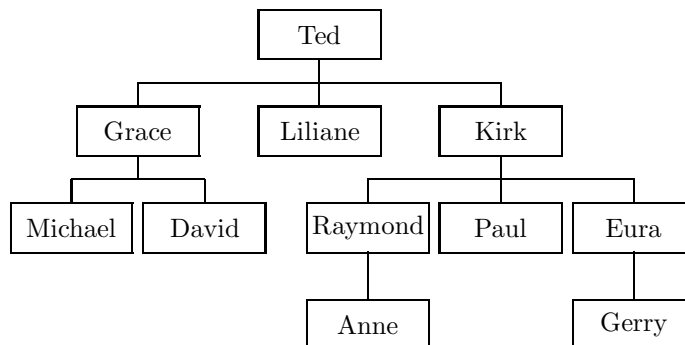
In any recursive algorithm, it is crucial that we have a way of stopping the process. This is often called a *terminating condition* of a recursive algorithm. In Example 1, the terminating condition was the case in which a person had no children. In Example 2, the terminating condition was the arrival at the core of the onion.

Exercises 11.1

1. The following is a recursive definition of the process of climbing a set of stairs.

```
To CLIMB THE STAIRS
  if you are at the top
    you are done
  else
    take one step and
    CLIMB THE STAIRS
```

- (a) Identify the simple case in `CLIMB THE STAIRS`.
 - (b) Identify the complex case in `CLIMB THE STAIRS`.
2. Why is the terminating condition so important in a recursive algorithm?
3. An old children's rhyme begins: "There once was a man who had a hollow tooth. In the tooth was a treasure chest. In the treasure chest was a piece of paper. On the piece of paper was written 'There once was a man who had a hollow tooth ...' ".
Is this an example of recursion? Justify your answer.
4. Which of the following can be said to be examples of recursion?
- (a) closing a zipper
 - (b) nested Russian wooden dolls
 - (c) a reflection of a mirror in a mirror
 - (d) the song that begins "There's a hole in my bucket ..."
5. Rewrite the definition of `CLIMB THE STAIRS` in Question 1 using a `while` statement.
6. Give a recursive description of the process of reading a book.
7. In the text, we gave recursive algorithms for the tasks of drawing a family tree and finding the centre of an onion. Find another task that can be defined recursively and give the recursive algorithm for it.
8. The diagram shows a family tree. If we were to draw this tree using the recursive algorithm of Example 1, in what order would the members of the family be inserted in the tree? (Assume that the children of any individual are added from left to right. For example, Michael would be added before David.)



11.2 Recursion in Mathematics

Suppose that we are given the following sequence of integers:

$$3, 6, 12, 24, \dots$$

in which any term is twice as large as the preceding one. If we call the terms $t_1, t_2, t_3, \dots$ then we can express the relationship concisely as follows:

$$\begin{aligned} t_1 &= 3 \\ t_n &= 2 \times t_{n-1}, \text{ if } n > 1 \end{aligned}$$

This definition of the terms of the sequence satisfies the conditions that we set out for any recursive definition:

1. An object is defined in terms of another object of the same type. Here a term is defined in terms of the preceding term.
2. There is some way of stopping the recursion. This is done here by explicitly defining the first term, t_1 , as having the value 3.

Example 1

Given the sequence defined recursively by the equations

$$\begin{aligned} t_1 &= 2 \\ t_n &= 3 \times t_{n-1} + 1, \text{ if } n > 1 \end{aligned}$$

find the value of the fifth term, t_5 .

From the definition, we know that

$$\begin{aligned} t_5 &= 3 \times t_4 + 1 \\ \text{and } t_4 &= 3 \times t_3 + 1 \\ \text{and } t_3 &= 3 \times t_2 + 1 \\ \text{and } t_2 &= 3 \times t_1 + 1 \\ \text{and } t_1 &= 2 \end{aligned}$$

Having found the value of t_1 , we can now substitute back into the expressions for the other terms.

$$\begin{aligned} t_2 &= 3 \times 2 + 1 = 7 \\ \text{and } t_3 &= 3 \times 7 + 1 = 22 \\ \text{and } t_4 &= 3 \times 22 + 1 = 67 \\ \text{and } t_5 &= 3 \times 67 + 1 = 202 \end{aligned}$$

The value of the fifth term is 202.

One of the most famous algorithms known was stated over two thousand years ago by the Greek mathematician Euclid. *Euclid's algorithm*, as it is called, provides a method of finding the greatest common divisor (gcd) of a pair of natural numbers. The algorithm is based on the following properties of a gcd:

Rule 1. The gcd of two equal values, m and n , is simply m (or n).

Rule 2. If two numbers are not equal, then the gcd of the two values will also divide their difference and will, in fact, be the gcd of this difference. Thus, if $m > n$, then the gcd of m and n will be equal to the gcd of n and $m - n$.

Rule 3. If the numbers are not equal and the first number is smaller than the second, then we can simply switch the numbers and apply Rule 2.

These three rules can be stated more concisely (and possibly more clearly) in symbolic form:

Rule 1. If $m = n$, then $\gcd(m, n) = m$

Rule 2. If $m > n$, then $\gcd(m, n) = \gcd(n, m - n)$

Rule 3. If $m < n$, then $\gcd(m, n) = \gcd(n, m)$

Example 2

Use Euclid's algorithm to find the greatest common divisor of 54 and 90.

$$\begin{array}{lll}
 \gcd(54, 90) & = & \gcd(90, 54) & \text{by Rule 3} \\
 & = & \gcd(54, 36) & \text{by Rule 2} \\
 & = & \gcd(36, 18) & \text{by Rule 2} \\
 & = & \gcd(18, 18) & \text{by Rule 2} \\
 & = & 18 & \text{by Rule 1}
 \end{array}$$

Exercises 11.2

- Write the first five terms of each sequence (correct to two decimal places).

(a) $t_1 = 3$

$$t_n = t_{n-1} + 2, \quad n > 1$$

(b) $t_1 = 2$

$$t_{n+1} = 1 - \frac{1}{t_n}, \quad n > 0$$

(c) $t_1 = 2$

$$t_{n+1} = n + t_n, \quad n > 0$$

(d) $t_1 = 1$

$$t_2 = 3$$

$$t_n = t_{n-1} + t_{n-2}, \quad n > 2$$

(e) $t_n = \begin{cases} 1 & \text{if } n = 1 \\ \sqrt{1 + t_{n-1}} & \text{if } n > 1 \end{cases}$

(f) $t_n = \begin{cases} 2 & \text{if } n = 1 \\ \sqrt{3t_{n-1} + 4} & \text{if } n > 1 \end{cases}$

- Write a possible recursive definition of each sequence.

- (a) 10, 13, 16, 19, ...
 - (b) $\frac{1}{2}, \frac{3}{2}, \frac{5}{2}, \dots$
 - (c) 5, 15, 45, 135, ...
 - (d) $-1, -3, -5, \dots$
 - (e) 256, 64, 16, 4, ...
 - (f) 1.1, 1.2, 1.3, 1.4, ...
3. Use Euclid's algorithm to find the gcd of each pair of numbers.
- (a) 18 and 24
 - (b) 84 and 144
 - (c) 35 and 16
 - (d) 515 and 206
4. A frog, sitting at one end of a two-metre log, jumps toward the other end but can only make it half way. The frog continues jumping toward the end but each time becomes more exhausted and only jumps half the length of the preceding jump.
- (a) State the lengths of the first four jumps.
 - (b) State an expression for the length in metres of the n^{th} jump, j_n , in terms of the $(n-1)^{\text{th}}$ jump, j_{n-1} .
 - (c) Use the recursive relationship established in part b) to find the distance jumped in the sixth jump.
 - (d) Give a non-recursive expression for j_n , the length in metres of the n^{th} jump.
5. (a) If we were to use Euclid's algorithm, as given in the text, to find the gcd of 44 and 6, how many subtractions would be required to reduce the problem to that of finding the gcd of 6 and 2?
- (b) What operation would achieve the same effect as this sequence of subtractions?
 - (c) Suggest a modification of the algorithm that avoids this sequence of subtractions.
 - (d) Use the modified algorithm to find the gcd of each pair of numbers in Question 3.
6. Prove that Euclid's algorithm, as stated in the text, correctly finds the gcd of two positive integers, m and n .

11.3 Recursive Queries

Suppose that we are given the following recursive definition of a sequence

$$\begin{aligned}t_1 &= 2 \\ t_n &= 3t_{n-1} - 1, \quad n > 1\end{aligned}$$

We can express such a sequence using function notation if we write

$$\begin{aligned}t(1) &= 2 \\ t(n) &= 3t(n-1) - 1, \quad n > 1\end{aligned}$$

To implement such a function with a Java method is simple. (The tricky part is understanding how the method operates.)

Example 1

The following method returns the value of the function defined by

$$\begin{aligned}t(1) &= 2 \\ t(n) &= 3t(n-1) - 1, \quad n > 1\end{aligned}$$

If the method is given an invalid value of the parameter, `n`, it throws an exception.

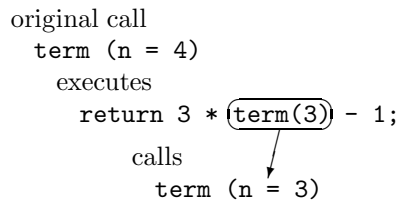
```
public static int term (int n)
{
    if (n < 1)
        throw new RuntimeException("Invalid parameter");
    else if (n == 1)
        return 2;
    else
        return 3 * term(n-1) - 1;
}
```

At first glance, the method may not appear to do anything; it seems to be nothing more than a definition of the sequence. To see how it works, suppose that `term` is called with `n = 4`. Since `n > 1`, the method executes the statement

```
return 3 * term(3) - 1;
```

The right side of this assignment statement involves a call to the method `term`. We have seen methods call other methods before. In such a case, execution of the first method is suspended until the called method returns the value it has been asked to compute. The same thing happens here; the only difference is that here both the calling method and the called method are the same. To visualize the process, we can think of a new copy of `term` being created by the call. This copy is identical to the original but it has its own copy of the parameter `n`. This version of the parameter `n` has the value 3. If the method had any local variables, there would also be separate copies of them in both the original and the called versions of the method.

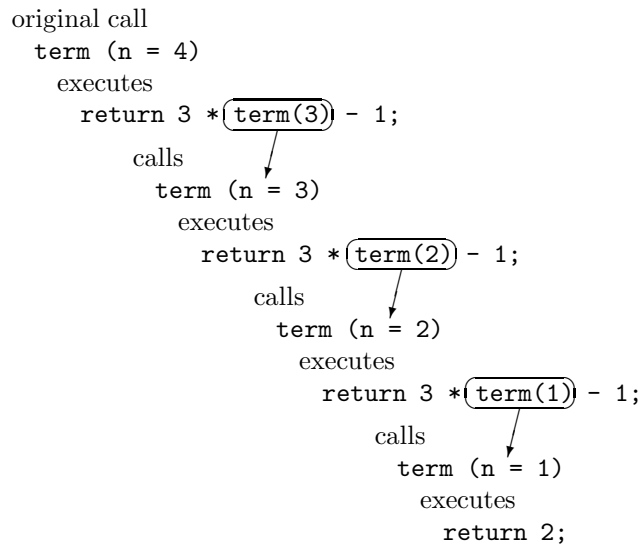
We can illustrate this process using a diagram.



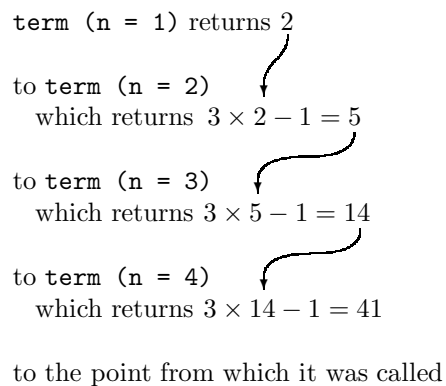
This copy of `term` (with `n = 3`) now executes the statement

```
term = 3 * term(2) - 1;
```

Before it can complete execution of this statement, it must call another copy of `term`, this time with `n = 2`. The process continues until a copy is called with `n = 1`, as shown in the next diagram.



The copy of `term` with `n = 1` now returns the value 2 to the point from which it was called and then ceases to exist. Knowing the value of `term(1)`, the value of the expression `3 * term(1) - 1` can now be completed; it gives $3 \times 2 - 1 = 5$. This value can then be returned to the point at which this version of `term` was called. This process continues until the original call computes and returns the desired value. Once again, we can illustrate the process with a diagram.



A silly but possibly useful way of looking at this process is to imagine yourself trying to find the value of $t(4)$. Rather than doing all the calculations yourself, you phone your friend Alex. Alex knows the formula for $t(n)$, so he can see that

$$t(4) = 3 \times t(3) - 1.$$

Unfortunately, he doesn't know $t(3)$ so he puts you on hold while he calls his friend Kim. Kim also knows the formula so she can quickly determine that

$$t(3) = 3 \times t(2) - 1.$$

To help her complete her calculations, Kim must have the value of $t(2)$. To get this, she puts Alex on hold while she phones her friend Chris. Chris also knows the formula so he can see that the answer to Kim's problem is

$$t(2) = 3 \times t(1) - 1$$

Like the other callers before him, Chris doesn't want to do all the work so he puts Kim on hold while he phones his friend Wei Ting to find $t(1)$. Wei Ting can see from the formula that

$$t(1) = 2$$

Wei Ting tells Chris that the answer is 2 and then hangs up (wondering why Chris would phone to ask her such a question).

Chris can now complete his calculations:

$$t(2) = 3 \times 2 - 1 = 5$$

Chris passes this information back to Kim and then he too hangs up.

Kim can now determine that:

$$t(3) = 3 \times 5 - 1 = 14$$

Kim tells Alex her result and then she also hangs up.

Alex can now complete his calculation:

$$t(4) = 3 \times 14 - 1 = 41$$

He passes this back to you and hangs up, leaving you to do what you want with the knowledge that

$$t(4) = 41$$

If all of this calling and returning seems to you to be a waste of time, you are right! Recursion is a very useful technique but it does *not* provide the most efficient way of evaluating a function like the one that we have here. We have shown this example only because it illustrates some aspects of recursion — not because we recommend that you use it in such circumstances.

Example 2

An efficient method for calculating the value of the function of the previous example would use a loop. Here is an efficient, non-recursive method that does the job.

```
public static int term (int n)
{
    if (n < 1)
        throw new RuntimeException("Invalid parameter");
    else
    {
        int value = 2;
        for (int i = 2; i <= n; i++)
            value = 3 * value - 1;
        return value;
    }
}
```

Recursive methods in Java are not limited to evaluating terms of sequences. They can be used to evaluate any function for which we have a recursive definition.

Example 3

A more efficient form of Euclid's algorithm than the one given in the previous section replaces the successive subtractions by one mod operation (implemented by the % operator in Java). The revised algorithm for determining the gcd of two non-negative integers is

$$\text{gcd}(m, n) = \begin{cases} m & \text{if } n = 0 \\ \text{gcd}(n, m \bmod n) & \text{if } n > 0 \end{cases}$$

The algorithm can be implemented easily in Java, as shown in the following method. (The method assumes that the values of both parameters are non-negative integers.)

```

public static int gcd (int m, int n)
{
    if (n == 0)
        return m;
    else
        return gcd(n, m % n);
}

```

If we make a call to `gcd(24,18)`, we produce the following sequence of actions:

```

original call
gcd (m = 24, n = 18)
executes
return (gcd(18, 24 % 18));
      |
      v
calls
gcd (m = 18, n = 6)
executes
return (gcd(6, 18 % 6));
      |
      v
calls
gcd (m = 6, n = 0)
executes
return 6;

```

Now, each copy of `gcd` returns, as shown in the next diagram.

```

gcd (m = 6, n = 0)
returns 6
to gcd (m = 18, n = 6)
which returns 6
to gcd (m = 24, n = 18)
which returns 6
to the point from which it was called

```

Exercises 11.3

- Trace the execution of the method `gcd` in Example 2 to find the greatest common divisor of each pair of values.

(a) <code>m = 20</code> <code>n = 28</code>	(b) <code>m = 991</code> <code>n = 129</code>
---	---

2. Trace the execution of the method `term` shown below, given an initial parameter value of 3.

```
public static double term (int n)
{
    if (n < 0)
    {
        System.out.println("Invalid argument to method"
                           + " term\nzero returned");
        return 0;
    }
    else if (n == 0)
        return 1;
    else
        return 2 + 0.5 * term(n-1);
}
```

3. What happens if the function `term` in Example 1 is called with a parameter whose value is zero?
4. What is wrong with these methods?

(a)

```
public static double bad (double a, double b)
{
    a = a/2;
    b = b*2;
    return bad(a,b);
}
```

(b)

```
public static int badToo (int n)
{
    if (n < 1)
        return 0;
    else if (n == 1)
        return 5;
    else return 2*badToo(n+1) + 3;
}
```

5. The *factorial* of a non-negative integer n (written as $n!$), is a function that is useful in many branches of mathematics. It can be defined recursively as follows:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1)! & \text{if } n > 0 \end{cases}$$

- (a) Write a recursive method **factorial** that returns the value of $n!$ as a **long** value. (A **long** can represent $n!$ for values of n up to 20.) If an invalid parameter value is given to the method, it should print an error message and return zero.
- (b) Write a non-recursive version of **factorial**.
- (c) Which version do you think would be more efficient? Justify your answer.
6. A function f is defined for integers x and y as follows:

$$f(x, y) = \begin{cases} -f(y, x) & \text{if } x < y \\ 0 & \text{if } x = y \\ 1 + f(x-1, y) & \text{if } x > y \end{cases}$$

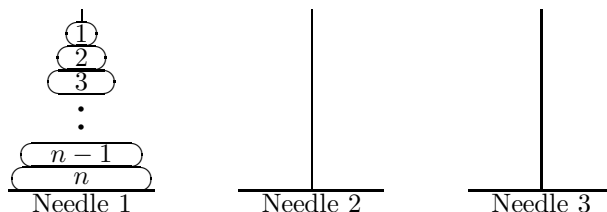
- (a) Use this definition to evaluate
 i) $f(5, 3)$ ii) $f(2, 2)$ iii) $f(1, 4)$ iv) $f(-5, -2)$
- (b) State, in a few words, the effect of the function f if it has integer arguments x and y .
7. A function that is important in the study of the theory of computation (but quite useless otherwise) is known as *Ackermann's function*. It can be defined for non-negative integers m and n as follows:

$$a(m, n) = \begin{cases} n + 1 & \text{if } m = 0 \\ a(m-1, 1) & \text{if } m > 0 \text{ and } n = 0 \\ a(m-1, a(m, n-1)) & \text{otherwise} \end{cases}$$

- (a) Evaluate $a(1, 1)$ and $a(2, 1)$ by hand.
- (b) Write a Java implementation of Ackermann's function.
- (c) Try to use your implementation to evaluate $a(5, 5)$. Account for the result.

11.4 Recursive Commands

Recursion is not limited to queries that return values. It can also be used with methods that perform some action and then simply return to the point at which they were called. To illustrate this, we will solve a problem usually known as the *Towers of Hanoi*. In this problem, a number of disks of decreasing size forms a “tower” on one of three needles. The smallest disk is at the top of the tower and the largest is at the bottom, as shown in the diagram.



The problem is to move all of the disks from Needle 1 to Needle 3 (using Needle 2 as necessary) subject to the following restrictions:

1. Only one disk may be moved at a time.
2. A disk may not be placed on top of a smaller disk.

If there is only one disk on Needle 1, then the problem is trivial as we can simply move the disk from Needle 1 to Needle 3. We will write this move in the form $1 \longrightarrow 3$.

A tower with two disks can be thought of as a one-disk tower placed on top of a larger disk. The instructions to move such a tower could be phrased in the following form:

- Move the one-disk tower from Needle 1 to Needle 2. ($1 \longrightarrow 2$)
- Move the bottom disk from Needle 1 to Needle 3. ($1 \longrightarrow 3$)
- Move the one-disk tower from Needle 2 to Needle 3. ($2 \longrightarrow 3$)

Similarly, a tower with three disks can be thought of as a two-disk tower placed on top of the largest disk. The instructions for moving this tower from Needle 1 to Needle 3 could then take the form:

- Move the two-disk tower from Needle 1 to Needle 2. This is a problem that we can solve using the procedure described above for a two-disk tower.

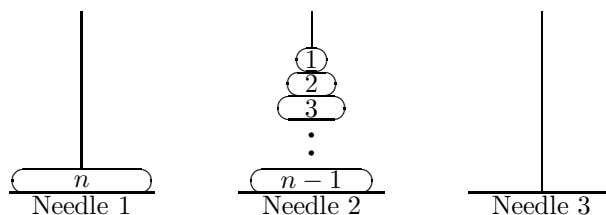
The solution is: $(1 \rightarrow 3 \quad 1 \rightarrow 2 \quad 3 \rightarrow 2)$

- Move the bottom disk from Needle 1 to Needle 3 $(1 \rightarrow 3)$
- Move the two-disk tower from Needle 2 to Needle 3. This is also a problem that we can solve using the procedure described above for a two-disk tower.

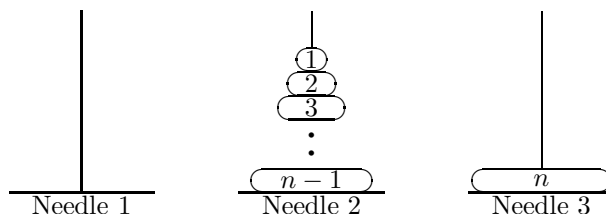
The solution is: $(2 \rightarrow 1 \quad 2 \rightarrow 3 \quad 1 \rightarrow 3)$

The pattern should now be clear. If there are n disks on Needle 1, then we can think of this as a tower of height $n - 1$ placed on top of the largest disk. To move the tower to Needle 3, we proceed as follows:

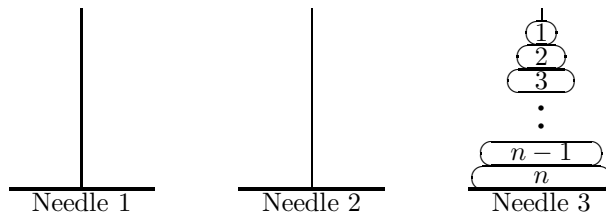
- Move the $(n - 1)$ -disk tower from Needle 1 to Needle 2.



- Move the bottom disk from Needle 1 to Needle 3.



- Move the $(n - 1)$ -disk tower from Needle 2 to Needle 3.



The second part of the solution is trivial; it simply requires the moving of one disk. The first and third parts, while they do not have obvious solutions if n is large, are problems that have the same form as the original problem except that they require that we move a tower that is smaller (by one disk) than the original. This is exactly what we want for a recursive solution — the solution to the original problem (moving a tower of n disks) has been stated in terms of twice solving a simpler problem of the same type (moving a tower of $n - 1$ disks).

The preceding analysis suggests (we hope) the following algorithm:

```

To MOVE A TOWER of  $n$  disks
  if  $n = 1$ 
    simply move the disk
  else
    MOVE A TOWER of  $n-1$  disks from the first needle
                                to the intermediate needle
    move the bottom disk from the first needle
                                to the last needle
    MOVE A TOWER of  $n-1$  disks from the intermediate needle
                                to the last needle

```

We use this algorithm in the recursive method `moveTower` that follows. The method's three parameters represent the height of the tower to be moved, the number of the needle on which the disks start, and the number of the needle on which the disks finish. The method prints the appropriate sequence of moves for such a tower. The method does not check for invalid parameter values; it assumes that the height is positive and that the start and finish needle numbers are two of the values 1, 2, or 3.

```

public static void moveTower (int height, int start,
                               int finish)
{

```

```

if (height == 1)
    // there is only one disk - simply move it
    System.out.println(start + " ---> " + finish);
else // solve the problem recursively
{
    // determine the other needle number using
    // the fact that the sum of the values of the
    // three needle numbers is always six.
    int other = 6 - (start + finish);

    // move all but one disk from start to other
    moveTower(height-1,start,other);

    // move the bottom disk
    System.out.println(start + " ---> " + finish);

    // move the remaining disks from other to finish
    moveTower(height-1,other,finish);
}
}

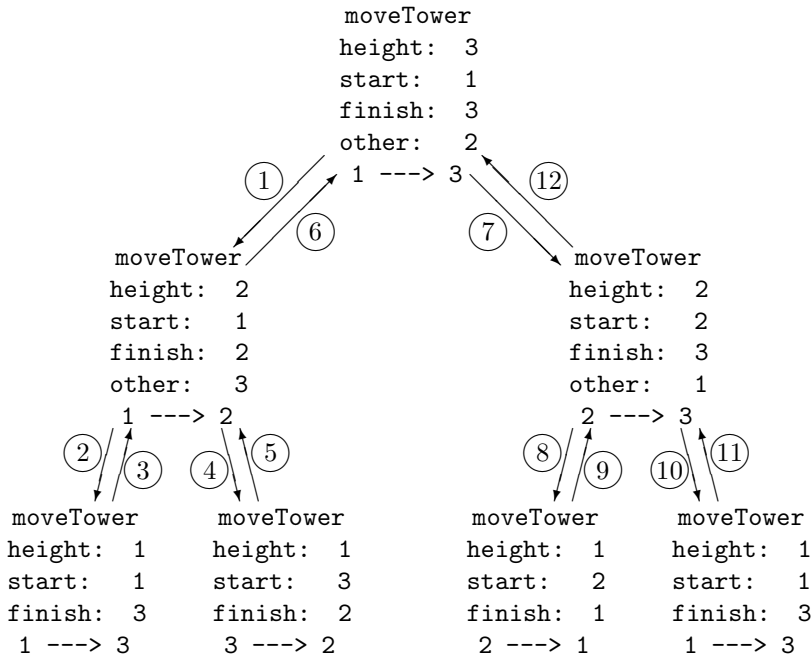
```

Example 1

If we were to call this method with the statement `moveTower(2,1,3)`; then the call would indicate that we want to move a tower of height two from Needle 1 to Needle 3. The call would produce the following output.

1	---	>	2
1	---	>	3
2	---	>	3

Tracing the action of a recursive method like `moveTower` in which there is more than one recursive call is, in general, not an easy task. A carefully drawn diagram, however, can often be helpful. The following diagram traces the action of `moveTower(3,1,3)`; The numbers shown in circles indicate the order in which the calls and returns would be performed.



The sequence of moves can be read from left to right:
1 → 3 1 → 2 3 → 2 1 → 3 2 → 1 2 → 3 1 → 3

Exercises 11.4

1. (a) In the Towers of Hanoi problem, the number of moves grows rapidly as the number of disks increases. By following the algorithm given in the text, complete the following table.

Number of Disks	Number of Moves
1	1
2	3
3	
4	
5	

- (b) Using the pattern of values in the table, predict the number of moves that would be required for
- i) 6 disks ii) 7 disks iii) 10 disks iv) n disks

- (c) The Towers of Hanoi problem is said to be based on a legend in which monks in a monastery near Hanoi came to be in possession of sixty-four golden disks mounted on one of three diamond needles. The monks' task was to attempt to move all the disks to another needle using the rules stated in the text. According to the story, after all the disks have been moved, the universe will come to an end. If we suppose that the monks began their task three thousand years ago and that they have been moving disks constantly at a rate of one per second ever since, find the approximate number of years from now to the time when the universe should end.
- 2. Trace the action of `moveTower(4,1,3)`;
- 3. (a) What will `moveTower` do if it is passed a zero or negative value for `height`?
(b) What will it do if it is given an invalid value of `start` or `finish`?
(c) Modify `moveTower` so that it will give an error message and attempt no moves if it is given invalid values for any of its parameters.
- 4. Write a program that uses `moveTower` to print the moves required to move n disks from Needle 1 to Needle 3 in the Towers of Hanoi problem. Your program should ask the user to give a positive value for the height of the tower (and should insist on being given a valid height before it starts to move the tower).
- 5. Study the method `convert` and then answer the questions that follow it.

```
public static void convert (int value, int base)
{
    if (value >= base)
        convert(value/base,base);
    System.out.print(value%base);
}
```

- (a) Trace the action of the method if it is called by the statement `convert(10,2)`;
- (b) Trace the action of the method if it is called by the statement `convert(22,3)`;

- (c) Describe in a few words the purpose of the method.
 - (d) What would the method do if called with `value < 0` and `base > 0`?
 - (e) Rewrite the method so that it works correctly even if `value` is negative.
6. (a) Give a recursive description of the process of printing a row containing n asterisks.
- (b) Complete the definition of a recursive method `printRow` whose header is shown below. The method should print a line containing a row of `n` asterisks. If `n` is less than one, the method should print nothing.
- ```
public static void printRow (int n)
```
7. Suppose that the following pattern is called a *5-triangle*.

```
*
**


```

- (a) Give a recursive description of the process of printing an  $n$ -triangle.
- (b) Write a recursive method `printTriangle` with a single `int` parameter `n`. The method should print a triangle of asterisks like the one shown here but containing `n` rows. If `n` is less than one, the method should print nothing.
8. What will the method `mystery` do if called with the argument 12345?

```
public static void mystery (int n)
{
 if (n > 0)
 {
 System.out.println(n);
 mystery(n/10);
 }
}
```

9. What output will the following methods produce if originally called by the statement `q(4, '*')`?

```
public static void p (int n, char symbol)
{
 if (n > 0)
 {
 System.out.print(symbol);
 p(n-1,symbol);
 }
 else
 System.out.println();
}

public static void q (int n, char symbol)
{
 if (n > 0)
 {
 p(n,symbol);
 q(n-1,symbol);
 }
}
```

## 11.5 Recursion with Strings

Although the applications of recursion that we have seen up to now are primarily concerned with numerical values, recursion can also be used to solve non-numerical problems. In this section, we will explore some uses of recursion in manipulating strings.

To develop recursive algorithms for strings, we must think of a non-empty string as a recursive structure. We can do this by considering a non-empty string as a character concatenated with a shorter string. An empty string, one with no characters, serves as a simple case.

## Example 1

To reverse the characters in a string, we could use the following algorithm.

```
To REVERSE a string
 if the string is non-empty
 place the first character at the end of
 the REVERSE of the tail of the string
```

For the simple case, an empty string, the reversal is simply the empty string itself. Here is a Java method that implements the algorithm. Given any string, the method will return the reversal of that string.

```
public static String reverse (String s)
{
 if (s.length() > 0)
 return reverse(s.substring(1)) + s.charAt(0);
 else
 return "";
}
```

A *permutation* of a set of items is an arrangement of those items. We can use recursion, along with a loop, to find all permutations of the characters of a string. All permutations can be formed by placing each character, in turn, in the first position of the string and following it with all possible permutations of the other characters. For example, if the string has the value "abcd", we can find its permutations systematically by taking

```
"a" followed by all permutations of "bcd"
"b" followed by all permutations of "acd"
"c" followed by all permutations of "abd"
"d" followed by all permutations of "abc"
```

The process can be continued recursively. At any point in the string, we can generate all permutations of characters from that point to the end by placing each character at the beginning of that substring and following it with all possible permutations of the remaining characters. To make this slightly more formal, we can state our algorithm as follows:

```
To PRINT STRINGS WITH ALL PERMUTATIONS of a string
 if we are at the end of the string
 print the current permutation
```

```

else
 for each character in the string from here to the end
 PRINT STRINGS WITH ALL PERMUTATIONS of remaining
 characters starting with that character

```

The next example shows a Java implementation of the algorithm.

## Example 2

The method that follows will print all permutations of the string `s` that can be obtained by rearranging the characters from position `index` to the end of the string. We can use it to find *all* permutations of a string by calling it with `index` equal to zero.

```

public static void permute (String s, int index)
{
 if (index == s.length())
 System.out.println(s);
 else
 for (int i = index; i < s.length(); i++)
 permute(s.substring(0,index)+s.charAt(i)
 +s.substring(index,i)+s.substring(i+1),index+1);
}

```

The inconvenience of having to specify a starting point for the permutation can be avoided. To do so, we can overload `permute` by writing two versions. The first version could take the following form:

```

public static void permute (String s)
{
 permute(s,0);
}

```

The second version would take the form seen in Example 2, with a second parameter denoting the current position in the string. The second version, called a *helper* method, would then call itself recursively to print all permutations of the characters. A user does not need to worry about specifying the starting point for the permutation required by the helper method. In fact, by making the helper method `private`, we can actually prevent a user from using the recursive version directly.

### Example 3

The pair of methods that follow, both called `permute`, work together to find and print all the permutations of a string.

```
public static void permute (String s)
{
 permute(s,0);
}

private static void permute (String s, int index)
{
 if (index == s.length())
 System.out.println(s);
 else
 for (int i = index; i < s.length(); i++)
 permute(s.substring(0,index)+s.charAt(i)
 +s.substring(index,i)+s.substring(i+1),index+1);
}
```

If we were to place the methods in a class called `Utilities`, then, to print all permutations of the string "abcd", we could write

```
Utilities.permute("abcd");
```

### Exercises 11.5

1. In Example 1, we developed and implemented an algorithm for reversing a string. Another approach to this problem is to proceed as follows:

```
To REVERSE a string
 if the string is non-empty
 place the last character at the front of
 the REVERSE of the rest of the string
```

Write a recursive method `reverse` that implements this algorithm. The method should have a header identical to that in Example 1.

2. Yet another way to create the reversal of a string is to place the reversal of the second half in front of the reversal of the first half. As an example, to obtain the reversal of the string "turnabout", we would concatenate the reversal of "bout" to 'a' to the reversal of "turn".

- (a) State this algorithm by completing the following, using the style shown in the previous question.

To REVERSE a string ...

- (b) Write a recursive method `reverse` that implements this algorithm. The method should have a header identical to that in Example 1.
3. Assuming that the methods shown in Example 3 are in the class `Utilities`, determine the output that would be produced by the statement
- ```
Utilities.permute("cat");
```

11.6 Backtracking Algorithms

Many experiments in psychology involve rats or other animals attempting to travel through a maze in order to receive some reward. On the assumption that any task that can be performed by a rat should also be within our grasp, we now examine the problem of programming the solution to a maze problem.

To represent our mazes, we will use objects containing rectangular arrays of characters in which the symbol '.' represents an open path, 'W' represents a wall, and 'X' represents an exit.¹ A typical maze array is shown here.

```
W W W W W W W W W W
W . . . W . W . . . W
W . W . W . W . W W W
W . W . . . . . W
W . W W W W . W W . W
W . W . . . . W . W
W W W W W W X W W W
```

¹In Chapter 13 we examine techniques for producing much better graphical displays but this will do for now.

Our problem is, given a starting point in the maze, to find an exit point, if there is one accessible from the given starting point. If the search for an exit is successful we also want to be able to see the path that has been found.

If we look at the problem from the rat's point of view, we can establish a recursive solution. The simplest case is finding ourselves at an exit. In this case, there is nothing more to do. If we are not at an exit, then we can try to find one by moving to an adjacent position that we have not yet tried and seeking an exit from there. (The rat can probably tell by smell which of the adjacent open positions have not been tried; we may have to use some other device.) If there are no open positions adjacent to our current one, then we must back up along the path that we took to get to this spot. This is where the idea of *backtracking* arises. If we ever get backed up to our original position without having found an exit and with no further paths to try, then we must give up — no exit can be reached from our starting point.

To enable us to keep track of the locations that we have tried, we use the character '-' which we call TRIED. Whenever we visit a location for the first time, we place a TRIED character in that spot. If we ever find a path that leads to an exit, we mark the locations along that path with the character '+' which we call GOOD_PATH. If an exit is found from an initial location, we place a sequence of GOOD_PATH characters in the maze, from the starting location to the exit.

Example 1

We can begin to define a `Maze` class as follows:

```
class Maze
{
    private static final char WALL = 'W';
    private static final char EXIT = 'X';
    private static final char OPEN = '.';
    private static final char TRIED = '-';
    private static final char GOOD_PATH = '+';
    private char[][] map;
}
```

To complete the `Maze` class, we need a constructor that creates a new maze and a method that prints the contents of a `map`. You are asked to write these methods in the exercises. The key method required by the `Maze` class is one that attempts to find the appropriate sequence of moves to an exit from a given location. The next example contains the code for a recursive method that performs this task.

Example 2

The method `findExitFrom` that follows attempts to move from the starting point in the array `map` whose indices are given by the parameters `row` and `col`. The method assumes that the perimeter of the `map` array contains only `WALL` or `EXIT` symbols. It also assumes that the position specified by `row` and `col` initially contains either an `OPEN` or `EXIT` symbol.

The method first checks to see if the current location is an exit and, if it is, the method returns the value `true`. Otherwise, it marks the current position with the symbol `Maze.TRIED`. This ensures that the method will not wander around the maze forever if it is forced to back up. The method then tries to move to an adjacent position in the maze. If the search for an exit from an adjacent point proves to be successful, the method marks the current position with the symbol `Maze.GOOD_PATH` indicating that this position is on a path that leads to an exit.

```
public boolean findExitFrom (int row, int col)
{
    boolean successful = false;
    if (map[row][col] == Maze.EXIT)
    {
        map[row][col] = Maze.GOOD_PATH;
        successful = true;
    }
    else
    {
        map[row][col] = Maze.TRIED;
        // try moving south
        if (map[row+1][col] == Maze.OPEN
            || map[row+1][col] == Maze.EXIT)
            successful = findExitFrom(row+1,col);
        if (!successful)
            // could not find exit in south - try moving east
```

```

        if (map[row][col+1] == Maze.OPEN
            || map[row][col+1] == Maze.EXIT)
            successful = findExitFrom(row,col+1);
    if (!successful)
        // could not find exit in east - try moving north
        if (map[row-1][col] == Maze.OPEN
            || map[row-1][col] == Maze.EXIT)
            successful = findExitFrom(row-1,col);
    if (!successful)
        // could not find exit in north - try moving west
        if (map[row][col-1] == Maze.OPEN
            || map[row][col-1] == Maze.EXIT)
            successful = findExitFrom(row,col-1);
    if (successful)
        // mark this as part of a correct path
        map[row][col] = Maze.GOOD_PATH;
    }
    return successful;
}

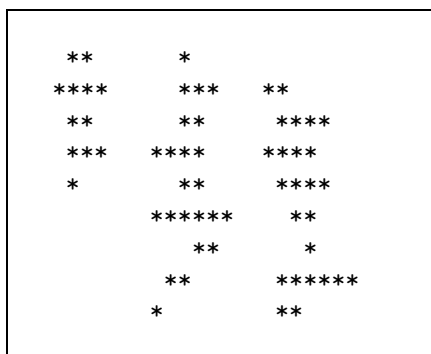
```



Exercises 11.6

1. Trace the path that would be taken by the method `findExitFrom` if it were given the maze shown in the text, starting at `map[1][8]`. Give your answer as a sequence of coordinates representing the row and column of each location visited — from the starting position (1, 8) to the exit position (6, 7).
2. Show the contents of the array `map` after the maze has been solved by the method `findExitFrom`.
3. Repeat the previous question if the method `findExitFrom` tried to move to an adjacent position in the order: north, west, south, east.
4. Write an instance method `print` for the `Maze` class. The method should print the `map` array in rectangular form.

5. Write a constructor for the `Maze` class that takes two parameters: the width and height of the array. The constructor should prompt the user for the contents of the array, one row at a time, as a sequence of strings. If a user provides a string that is invalid, the constructor should reject it and prompt, repeatedly if necessary, for another string for that row. For a maze to be valid, it must contain only the characters for a wall, an exit, or an open cell. In addition, the perimeter must contain only wall or exit characters.
6. Write a complete program that creates a maze, prints it, prompts the user for a starting position in the maze, attempts to find an exit from that point, and prints the resulting maze, along with a message indicating whether or not the search for an exit was successful.
7. Suppose that we are given a rectangular array containing blanks on the perimeter and either blanks or asterisks on the interior, as shown in the following diagram:



We say that two asterisks are *connected* if they are adjacent vertically, horizontally, or diagonally. We call a group of connected asterisks a *blob*. The diagram contains three blobs.

Consider the problem of starting at any point in a blob and erasing the entire blob, leaving any other blobs intact.

- (a) What is the simplest case of this problem?
- (b) Give a recursive algorithm that could be used to erase a blob.

11.7 Quicksort

When we last looked at sorts, the fastest one that we saw was Shellsort. Now, using recursion, we can examine an even faster algorithm, named *quicksort* by its inventor, C. A. R. Hoare.²

Quicksort, like all of our previous sorts, uses a number of passes to achieve its goal of ordering an entire list of values. Like insertion sort, quicksort inserts one more item into the list on each pass. However, unlike insertion sort, quicksort inserts items into their *final* positions. Once an item has been inserted by quicksort, it never moves again.

The sort is based on the following very simple idea. If a list is sorted (in ascending order, say) then, if we look at any value, K , all the values less than K will be on its left and all values greater than K will be on its right, as shown in the next diagram. (We assume for now that all values are unique; it is simple to adapt the sort to cases where there are repeated values.)

Values less than K	K	Values greater than K
----------------------	-----	-------------------------

A pass of quicksort chooses one value K in an unsorted list to be inserted in its correct location. This element is called a *pivot*. The pass then rearranges the other elements, if necessary, so that all values less than the pivot are on its left and all values greater than the pivot are on its right. The values other than K may still be out of order but K will surely be in the position that it will occupy once the list is fully sorted. We call such a rearrangement a *partition* of the list.

To achieve a partition, we will use two markers to locate values that are on the wrong end of the list and send them over to the correct end. Let us see how to partition a list by looking at an example.

Example 1

Suppose that we are given the following list of integers:

61	46	12	63	52	91	27	55	74	14	71	37	87
----	----	----	----	----	----	----	----	----	----	----	----	----

²Quicksort and, in fact, any recursively defined process, can be done without recursion but it is easier to describe this way.

We begin our first pass of quicksort by choosing one element to be the pivot. We arbitrarily choose the left hand element, 61. The pass now proceeds to divide the list so that values less than 61 are on the left end of the list, values greater than 61 are on the right end of the list, and 61 is between these two sets of values, in its final, correct, position. To do this, a copy is made of the pivot (61 in our case) and two markers, **left** and **right**, are set to point to the leftmost and rightmost locations in the list, as shown in the next diagram.

61	61	46	12	63	52	91	27	55	74	14	71	37	87
↑	↑												↑
pivot	left												right

The **right** marker starts moving toward the centre looking for values that should not be on the right. Since $87 > 61$, 87 is allowed to remain where it is. Since $37 < 61$, it should go to the left. To get it there, it is copied into the location indicated by the **left** marker, formerly occupied by 61. This gives

61	37	46	12	63	52	91	27	55	74	14	71	37	87
↑	↑											↑	
pivot	left											right	

Now the **left** marker starts moving toward the centre looking for values that should not be on the left because they are greater than the pivot. Since $46 < 61$, 46 is allowed to remain where it is. Since $12 < 61$, it is also allowed to stay in its location. Since $63 > 61$, it is copied into the location marked by **right**. This gives

61	37	46	12	63	52	91	27	55	74	14	71	63	87
↑				↑								↑	
pivot				left								right	

Again, **right** moves toward the centre, looking for values that are on the wrong end of the list. It finds 14 and sends it to the location marked by **left**, giving

61	37	46	12	14	52	91	27	55	74	14	71	63	87
↑				↑						↑			
pivot				left						right			

Now **left** moves toward the centre, finding 91 on the wrong side and sending it to the right.

61	37	46	12	14	52	91	27	55	74	91	71	63	87
↑						↑				↑			
pivot						left				right			

Again, **right** moves toward the centre, finding 55 on the wrong side and sending it to the left.

61	37	46	12	14	52	55	27	55	74	91	71	63	87
↑						↑		↑					
pivot						left		right					

Finally, **left** moves toward the centre and this time finds **right** before it detects any more values on the wrong side. The cell that both **left** and **right** meet at is the correct location for the pivot. It is copied into that spot to give the following arrangement at the end of the pass.

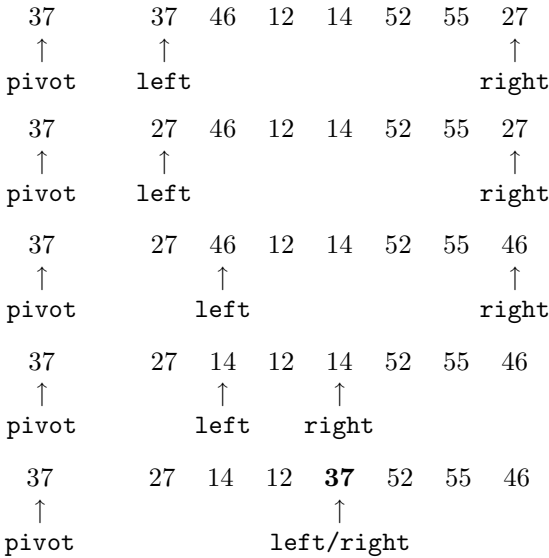
61	37	46	12	14	52	55	27	61	74	91	71	63	87
↑								↑					
pivot								left/right					

If all goes well on the first pass of quicksort, one value (the pivot) is inserted near the centre of the list with all lesser values on its left and all greater values on its right. As we said earlier, if the list values were completely sorted, the pivot would have exactly the same values on its left and on its right as it does after the partitioning. Thus the pivot must be in its final position after the first pass; we no longer have to worry about it. What we *do* have to worry about after the first pass are the two unsorted sublists on either side of the pivot. To take care of them, we apply our partitioning technique recursively on each sublist.

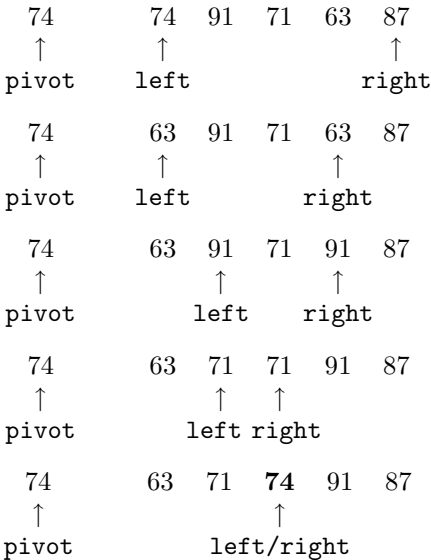
Example 2

Using the list produced in Example 1, let us see how the list would appear if the partitioning technique used there is applied to the sublists on the left and right side of the original pivot.

For the left sublist, we take the pivot to be its leftmost item, 37. The diagrams show the progress of the **left** and **right** markers as they seek out the correct position for 37 in the sublist.



For the right sublist, we have the following stages in the insertion of the pivot 74 in its correct position.



If all the partitions of Example 1 and Example 2 are performed, the list would then have the following appearance:

27 14 12 **37** 52 55 46 **61** 63 71 **74** 91 87

To completely sort a list, quicksort continues to partition sublists as long as the sublists contain more than one item. If a sublist contains fewer than two items, no further work needs to be done on it. A Java method to perform quicksort on an array of items of type `Object` is shown in the next example.

Example 3

The methods that follow, both called `quickSort`, work together to implement quicksort to sort an array of `Object` values. They work whether or not there are repeated items in the array. If we were to place the methods in a class called `Utilities` then, to perform a sort on an array called `myList`, a user would write

```
Utilities.quickSort(myList);
```

This would invoke the first method and it would then call the second, with the appropriate bounds for the first pass of the sort. The second version, a helper method, would then call itself recursively on the sub-intervals.

```
public static void quickSort (Object[] list)
{
    quickSort(list,0,list.length-1);
}

private static void quickSort (Object[] list,
                               int low, int high)
{
    final int MOVING_LEFT = 0;
    final int MOVING_RIGHT = 1;

    if (low < high)
    {
        int left = low;
        int right = high;
        int currentDirection = MOVING_LEFT;
        Object pivot = list[low];
        while (left < right)
```

```

{
  if (currentDirection == MOVING_LEFT)
  {
    while (list[right].compareTo(pivot) >= 0
           && left < right)
      right--;
    list[left] = list[right];
    currentDirection = MOVING_RIGHT;
  }
  if (currentDirection == MOVING_RIGHT)
  {
    while (list[left].compareTo(pivot) <= 0
           && left < right)
      left++;
    list[right] = list[left];
    currentDirection = MOVING_LEFT;
  }
}
list[left] = pivot; // or list[right], since equal
quickSort(list,low,left-1);
quickSort(list,right+1,high);
}
}

```



Does quicksort deserve its name? In general, it does. It is, on average, faster than any of the other sorts that we have examined in this text. It does, however, have its problems. For short lists, it is more trouble than it is worth; the bookkeeping involved in the recursive calls wastes too much time in this situation and it might be better to use Shellsort or (for very short lists) selection sort or insertion sort. Another problem arises if the partitions do not split the sublists near their midpoints. If the number of such bad partitions becomes very large, the efficiency of quicksort can be eroded very badly. A number of techniques have been found for avoiding bad partitions but they are beyond the scope of this book.

Exercises 11.7

1. Given the following list, show the stages in quicksort's partitioning of the list (as in Example 2) if the element 12 is used as the pivot and the list is to be sorted in ascending order.

12 7 20 6 21 18 2 14 15 5

2. Suppose that quicksort is used to sort 31 elements and suppose further that every partition produces sublists containing an equal number of elements.
 - (a) How many calls are made to the method `quickSort`?
 - (b) What is the total length of all lists passed to `quickSort`?
3. Repeat the previous question but now suppose that the list is already sorted before quicksort is called so that, at every partition, the pivot element stays at the left end of the list.
4.
 - (a) Compare the results of the preceding two questions.
 - (b) Can you suggest a modification in the way that the pivot is selected that would enable quicksort to operate efficiently even if the items are already ordered (or nearly so)?
5. Write a program that first generates 500 random integers in the interval $0 \dots 999$, prints the values, uses a version of quicksort to sort the values in descending order, and prints the sorted values.

11.8 Algorithm Analysis Revisited

In the previous chapter, we introduced the idea of analysis of the performance of algorithms and developed the use of big oh notation to describe this performance. Having introduced some new algorithms in this chapter, we can now extend these ideas.

In analyzing the behaviour of searching and sorting algorithms, we used the number of comparisons to obtain an estimate of the rate of growth

of time, T , as the size, n , of the problem increased. To analyze the behaviour of our algorithm to solve the Towers of Hanoi problem, we use the number of moves to obtain an approximation of the rate of growth of T as the number of disks, n , increases. Suppose that we let the number of moves required to move a tower of n disks be m_n . The recursive solution to the problem says that, to move a tower of height n , we first move a tower of height $n - 1$ out of the way, then move the bottom disk, and finally move the tower of height $n - 1$ to the final position. Thus

$$\begin{aligned} m_n &= m_{n-1} + 1 + m_{n-1} \\ &= 2m_{n-1} + 1 \end{aligned}$$

Similarly, to move a tower of height $n - 1$, we have

$$m_{n-1} = 2m_{n-2} + 1$$

Substituting the second equation into the first gives

$$\begin{aligned} m_n &= 2(2m_{n-2} + 1) + 1 \\ &= 2^2m_{n-2} + 2 + 1 \end{aligned}$$

Substituting $m_{n-2} = 2m_{n-3} + 1$ into this equation gives

$$\begin{aligned} m_n &= 2^2(2m_{n-3} + 1) + 2 + 1 \\ &= 2^3m_{n-3} + 2^2 + 2 + 1 \end{aligned}$$

After k steps, we have

$$m_n = 2^k m_{n-k} + 2^{k-1} + \cdots + 2^2 + 2 + 1$$

When $k = n - 1$, this gives

$$\begin{aligned} m_n &= 2^{n-1} m_{n-(n-1)} + 2^{n-1-1} + \cdots + 2^2 + 2 + 1 \\ &= 2^{n-1} m_1 + 2^{n-2} + \cdots + 2^2 + 2 + 1 \end{aligned}$$

Since $m_1 = 1$, this simplifies to

$$m_n = 2^{n-1} + 2^{n-2} + \cdots + 2^2 + 2 + 1$$

Summing the geometric series on the right side, we have, finally

$$m_n = 2^n - 1$$

For large values of n , this gives

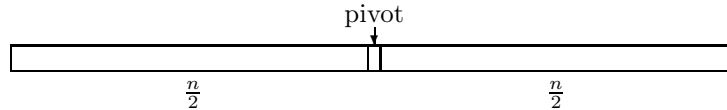
$$m_n \doteq 2^n$$

Since the time required to perform the algorithm will be roughly proportional to the number of moves, this gives us

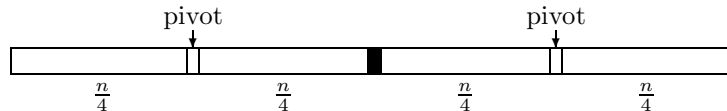
$$T \text{ is } O(2^n)$$

To analyze the performance of quicksort, we proceed as we did for the sorts in Chapter 10, by examining the number of comparisons. In our analysis, we will make the assumption that each partition splits its sublist into equal (or nearly equal) parts. As we noted when we were developing the quicksort algorithm, our version of quicksort can be modified fairly easily to guarantee that this will be the case.

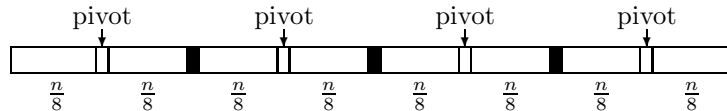
On our first pass, we examine approximately $\frac{n}{2}$ values on the left end of the list and $\frac{n}{2}$ values on the right end of the list, a total of $\frac{n}{2} + \frac{n}{2} = n$ comparisons.



We can then repeat the process on both halves of the list. In each half, we examine approximately $\frac{n}{4}$ values on each side of the pivot, a total of $\frac{n}{4} + \frac{n}{4} + \frac{n}{4} + \frac{n}{4} = n$ comparisons. (In counting comparisons, we ignore the fact that the old pivot is no longer being examined.)



Repeating the process in each of the four sub-lists requires the examination of approximately $\frac{n}{8} + \frac{n}{8} + \frac{n}{8} + \frac{n}{8} + \frac{n}{8} + \frac{n}{8} + \frac{n}{8} + \frac{n}{8} = n$ comparisons.



How many times must we repeat this process before we are done? At each stage, the size of each sublist is being cut (approximately) in half and the process continues until the size is reduced to one item. This is a situation that we have seen and analyzed previously, when we studied the behaviour of binary search. There we showed that $\lg n$ steps are required to reduce the list size to one. Here, however, at each of these $\lg n$ steps, we must perform approximately n comparisons. Thus quicksort will require a total of $n \times \lg n$ comparisons to completely sort a list. Hence, under our assumption that the pivots will be in the middle of their lists, quicksort is an $O(n \lg n)$ algorithm.

Although it is almost always true that, as a problem size gets larger, we have to do more work to solve the problem, this is not always the case. For example, the amount of work required to retrieve a value from an array does not depend on the size of the array. If the time required to perform an operation is no more than some constant, k , then we can write

$$T \text{ is } O(k)$$

Since we are dealing with proportionalities, we can divide by k to obtain the usual representation if the time required by an algorithm is bounded by some constant:

$$T \text{ is } O(1)$$

To give some feeling for the range of times required by various algorithms, the next table gives approximate values of a number of the functions that we have encountered in our analysis of algorithms.

n	$\lg n$	$n \lg n$	n^2	2^n
10	3	30	100	1 000
100	7	700	10 000	10^{30}
1 000	10	10 000	1 000 000	10^{300}
10 000	13	130 000	100 000 000	10^{3000}

A number of things can be deduced from an examination of the table. For example, it is obvious that, for large amounts of data, an algorithm like quicksort that is $O(n \lg n)$, is *far* better than an algorithm like selection sort that is $O(n^2)$. Also, we can see that $O(2^n)$ algorithms are hopelessly slow for all but the smallest values of n .

In comparing algorithms, we must keep in mind that big oh notation ignores constants. For small values of n , an algorithm for which the running time is $5n^2$ is preferable to one whose running time is $5000 n \lg n$. Only when n is large will this $O(n \lg n)$ algorithm outperform this $O(n^2)$ one.

Exercises 11.8

1. An algorithm whose running time is known to be $O(2^n)$ requires t s to process a problem of size n . About how long would it take (in the same computing environment) to process a similar problem of each of the given sizes?
 - (a) $n + 1$
 - (b) $n + 3$
 - (c) $2n$
 - (d) $3n$
2. Suppose that the running time of Algorithm A is $400n^2$ while the running time of Algorithm B is $0.2n^3$. For what values of n will Algorithm A run more quickly than Algorithm B?
3. In this section, we showed that the number of moves, m_n , to move a tower of height n in the Towers of Hanoi problem using our recursive algorithm, is $2^n - 1$. Use mathematical induction to prove this result.
4. Run an experiment to compare the running times of quicksort and selection sort. For the experiment, use arrays of random integers in the range $0 \dots 99999$. Use the method `currentTimeMillis` in the class `java.lang.System` to time the sorts. Experiment with various sizes of arrays, starting at 1000 and then doubling the length on each run until you get some useful results without using too many computer resources (either space or time).

11.9 Avoiding Errors and Debugging

Avoiding Errors

1. The most common error in using recursion is writing something that never terminates. This can be avoided by always checking that
 - (a) there is a simple case in which the method does not make a recursive call and
 - (b) any recursive call is a step *closer* to the simple case.
2. One way to avoid problems with recursion is to simply avoid using recursion at all. Although recursive solutions to problems are often more elegant than non-recursive ones, it is always possible to replace a recursive solution with a non-recursive one. For many problems, a non-recursive approach is more appropriate. There is a certain amount of overhead in using a recursive solution because every time a method is called recursively, time is used by the calling process and more space must be allocated for any parameters and local variables. If a *simple* iterative solution exists, it should be used in preference to a recursive one.

Debugging

1. If you happen to write something in which the recursion never terminates, the recursive method will continue to call copies of itself. Each of these calls requires space, to keep track of the values of parameters and local variables. Eventually, there will be no room left in the computer for any more calls. In such a case, execution of your program will likely terminate with a message about stack overflow. If you get such a message, this is the likely cause.
2. Start debugging recursive methods by testing the simple case first. Only after you are sure that this is working should you move on to testing with more complex cases.

3. Insert tracing statements to print the value of key variables each time the recursive method is called. If the program is experiencing problems with runaway recursion, these tracing statements will likely fly past your face on the screen when you run the program. To make the tracing statements readable, insert requests for input just after the values are printed.

Exercises 11.9

1. State the error in each method.

```
(a)    public static int f (int n)
        {
            if (n >= 0)
                return f(n-1) * n;
        }

(b)    public static double g (double x)
        {
            if (x > 5)
                return g(2*x) + 1;
            else
                return 7;
        }
```

2. Write a method (with a single `int` parameter `n`) that provides a non-recursive solution to the problem of computing the value of the n^{th} term of the sequence whose recursive definition follows. If $n < 1$, the method should return zero.

$$\begin{aligned} t_1 &= 2 \\ t_n &= 3t_{n-1} - 1, \text{ if } n > 1 \end{aligned}$$

3. Consider the following definition of a sequence.

$$\begin{aligned} t_1 &= 1 \\ t_2 &= 2 \\ t_3 &= 3 \\ t_n &= t_{n-1} + t_{n-3}, \text{ if } n > 3 \end{aligned}$$

- (a) Write the first six terms of the sequence.
- (b) Write a non-recursive method (with a single `int` parameter `n`) that returns the value of the n^{th} term of the sequence. If $n < 1$, the method should return zero.

11.10 Review Exercises 11

1. Give a recursive description of the process of eating a bag of chips.
2. Suppose that you have two unsorted piles of cards (A and B) with each card in each pile containing the title of a book. Your task is to find all titles that appear in both piles and write them on a piece of paper without sorting the cards in either pile. Give a recursive description of this process. You may assume that there are no duplicate titles in either pile.
3. (a) State, in a few words, the effect of the function f whose definition (for non-negative integers x and y) is

$$f(x, y) = \begin{cases} 0 & \text{if } y = 0 \\ x + f(x, y - 1) & \text{otherwise} \end{cases}$$

- (b) Write a recursive Java method that implements the function.
4. The *Fibonacci sequence* consists of the numbers

$$1, 1, 2, 3, 5, 8, \dots$$

and can be defined recursively as follows:

$$f(n) = \begin{cases} 1 & \text{if } n = 1 \\ 1 & \text{if } n = 2 \\ f(n - 1) + f(n - 2) & \text{if } n > 2 \end{cases}$$

- (a) Write a recursive method `fibonacci` with an integer parameter `n`. The method should return the n^{th} term of the Fibonacci sequence. (If $n < 1$, the method should produce an error message and return the value zero.)

- (b) How many calls are made to `fibonacci` in computing the value of the fifth term of the Fibonacci sequence?
 - (c) Write a non-recursive version of `fibonacci`.
 - (d) Which version of `fibonacci` is more efficient? Why?
5. Write a recursive method `digitSum` that has a single `int` parameter. The method should return the sum of the digits of the parameter.
6. Consider the pattern shown in the following diagram to be of size 4 because the longest row contains four asterisks.

```

*****
****
***
**
*
**
***
*****

```

- (a) Give a recursive description of the process of printing a similar pattern, of size n .
 - (b) Write a recursive method `printPattern` that will print a pattern like the one shown here. The method should have a single parameter `n` that specifies the length of the longest row in the pattern. If `n` is less than one, the method should print nothing.
7. The Tchebyshev polynomials can be defined as follows:

$$T_n(x) = \begin{cases} 1 & \text{if } n = 0 \\ x & \text{if } n = 1 \\ x \cdot T_{n-1}(x) - T_{n-2}(x) & \text{if } n > 1 \end{cases}$$

- (a) Find $T_2(x)$ and $T_3(x)$ in simplified form.
 - (b) Find the value of $T_4(2)$ by hand.
 - (c) Complete the definition of a method `tchebyshev` whose header is shown here. The method should return the value of $T_n(x)$. If $n < 0$, the method should return the value zero.
- ```

 public static double tchebyshev (int n, double x)

```

8. To compute the value of  $x^n$ , where  $n$  is a positive integer, a recursive solution can be obtained using the following relations:

$$x^n = \begin{cases} \left(x^{\frac{n}{2}}\right)^2 & \text{if } n \text{ is even} \\ \left(x^{\frac{(n-1)}{2}}\right)^2 \times x & \text{if } n \text{ is odd} \end{cases}$$

$$\begin{aligned} \text{For example, } 4^6 &= (4^3)^2 \\ &= ((4)^2 \times 4)^2 \end{aligned}$$

Once the expression has been factored, we can then evaluate it with only a few multiplications. Using the preceding example, we have

$$\begin{aligned} 4^6 &= ((4)^2 \times 4)^2 \\ &= (16 \times 4)^2 \\ &= (64)^2 \\ &= 4096 \end{aligned}$$

- (a) Use the given relationships to write each power in factored form.  
 i)  $x^{10}$                       ii)  $x^{25}$                       iii)  $x^{100}$
- (b) How many multiplications would be needed to evaluate each of the factored expressions found in part (a)?
- (c) Write a recursive method **power**, with **double** parameter **x** and **int** parameter **n**, that uses the given relationships to find the value of  $x^n$ . In your method, assume that  $n > 0$ .
- (d) Extend your method so that it gives correct answers for *any* integral exponent. (The method should return NaN as the value of  $0^0$ .)
9. In Section 10.2, we developed a method that used binary search to determine whether or not an item was in a sorted list. The method had the header
- ```
public static int binSearch(double[] list, double item)
```
- The method returns the index in **list** of **item** (if **item** is in **list**) and -1 otherwise. Rewrite the method so that it uses recursion to perform its task. To carry out the recursion, use a helper method with the header
- ```
private static binSearch (double[] list, double item,
 int low, int high)
```

10. A polynomial of the form

$$p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_ix^i + \cdots + a_nx^n$$

can be evaluated efficiently using a technique known as *Horner's rule*. Horner's rule avoids computing the numerous  $x^i$  values explicitly, since this is redundant and slow, by rewriting and evaluating  $p(x)$  in the following form:

$$p(x) = a_0 + x \times (a_1 + x \times (a_2 + \cdots + x \times (a_i + \cdots + x \times (a_n) \cdots) \cdots))$$

Complete the definition of the method `horner` with header

```
public static double horner (double[] a, double x)
```

so that it uses Horner's method to evaluate a polynomial. The first parameter is an array of coefficients of the polynomial, the second is the value of  $x$  at which the polynomial is to be evaluated. The method should use a recursive helper method with the header

```
private static double horner(double[] a, double x, int i)
```

The third parameter in the helper method is an index that you should use to keep track of your recursion.

11. Write a method `printSub` with an `int` parameter `sum` and an `int` array parameter `list`. The method should look for a sublist of `list` whose elements add up to `sum`. If it finds such a sublist, it should print it. For example, if `list = {3,7,5,3,8}` and `sum = 12`, then the method should print 7 and 5. You may assume that there will be no more than one sublist satisfying the condition.
12. The number five can be written as the sum of three positive integers in two different ways:  $5 = 1 + 1 + 3$  and  $5 = 1 + 2 + 2$ . (Different orderings of the same values are not to be counted.) Write a method with header
- ```
public static void partition (int n, int r)
```
- that uses a recursive helper method to print all the ways that the positive integer `n` can be written using `r` positive integers.

Projects

13. Although some equations can be solved directly using a formula, for many equations it is impossible to do so. For such equations we can, however, usually obtain approximations to the roots that are

arbitrarily close to the actual values. There are a number of methods that can be used to do this. To find a root of the equation $f(x) = 0$, we can begin by considering the related function $y = f(x)$ and try to find an x -intercept of this function. An x -intercept of the function is a root of the corresponding equation.

One such method is known as the *bisection method*. With this method, we start the process with an interval `[xLeft,xRight]` in which the function f is known to be continuous and in which the sign of $f(\text{xLeft})$ is different from the sign of $f(\text{xRight})$. Since the function is continuous and it changes sign in the interval, it must have an x -intercept somewhere in the interval.

We can find a smaller interval containing the root by first finding the midpoint, `xMid`, of the interval and then calculating the value of $f(\text{xMid})$. If the value of $f(\text{xMid})$ is exactly zero, then we have found the exact root of our equation. If it has a value whose sign is opposite to that of $f(\text{xLeft})$, then the root must lie between `xLeft` and `xMid`. If it has a value whose sign is opposite to that of $f(\text{xRight})$, then the root must lie between `xMid` and `xRight`. By repeating this process, we can find smaller and smaller intervals containing the root of the equation. The process can be terminated either by finding an exact root or by narrowing the interval in which the root must lie to be less than some specified value *epsilon*.

Your problem is to write a program that uses the bisection method *recursively* to solve a cubic equation of the form

$$ax^3 + bx^2 + cx + d = 0$$

Your program should begin by reading (as `double` values, each on a separate line) the left and right ends of an initial interval and a value of `epsilon`. The program should then use the bisection method to find a root of the equation in the interval specified by `epsilon`. Your program should continue to read values and find roots until the user supplies input consisting only of zeros. Assume that all input will be valid. Your program should consist of three methods in a single class:

- a method that calculates the value of a cubic function. The method should have a header of the form

```
public static double f (double x)
```

It should calculate and return the value of

$$f(x) = 2x^3 - 4x^2 - 10x + 15$$

- a recursive method that uses the bisection method to find an x -intercept of the cubic function (and hence a root of the corresponding equation)
 - a **main** method to do the reading of input, writing of output, and calling of the method to do the bisection
14. In the game of chess, the queen is the most powerful piece. It can attack another piece that lies on the same row, the same column, or the same diagonal as the queen. Despite this power, it is still possible to place many queens on a board at the same time so that none of them can attack any of the others. As an example, the diagram shows four queens distributed on a 4×4 board so that none threatens any other.

		Q	
Q			
			Q
	Q		

Your problem is to write a program that uses recursion with backtracking to attempt to place n queens on an $n \times n$ board. To solve the problem, it is useful to think of the board as consisting of n rows each containing n columns rather than simply a board of n^2 squares. Since a queen can move any number of squares vertically in a single move, there can be no more than one queen in each column. Since you are required to place n queens on the board, there must be exactly one queen in every column. A one-dimensional array of size n can be used to record an entire solution. Using the previous example, the arrangement of the four queens can be represented by the array

1	3	0	2
---	---	---	---

Using such an array, we can attempt to place the queens on the board from left to right. For each column, we attempt to find a safe row for a queen (one not under attack from previously placed queens), placing a queen there, and then making a recursive call to find a complete solution given that placement. To check that a placement is safe from attack by previously placed queens, it is easy to determine whether

or not it is in an occupied row. Checking to see that it is in a safe diagonal is trickier but it can be done efficiently and your program should do so. Your program should first prompt the user for a board size and then continue to attempt to place queens in columns until all columns have been assigned a queen or no safe position can be found for a queen in one of the columns. If a solution is found, the program should print the contents of the array indicating the positions of the queens.

15. Yet another sorting technique is the *merge sort*.³ As with quicksort, the algorithm for a merge sort can be stated most easily in recursive form, as follows:

```
To SORT a list of n items
  if (n > 1)
  {
    SORT the left half of the list
    SORT the right half of the list
    MERGE the two sorted halves
  }
```

The MERGE phase takes the two sorted halves of a list and combines them into one sorted list. For example, if, before a merge, a list contained

12	18	40	72	20	32	45	63
└──────────┘				└──────────┘			
sorted				sorted			

then, after the merge, the list would contain

12	18	20	32	40	45	63	72
└──────────────────────────┘							
sorted							

Merging is done non-recursively, using, essentially, only one pass through the list.

Write a recursive merge sort and then test it using arrays of randomly generated integers in the range 0 . . . 99999 and sorting them into ascending order. Use an insertion sort to perform the same task and compare the running times of the two sorts. Use the method `currentTimeMillis` in the class `java.lang.System` to time the sorts. Experiment with various sizes of arrays, starting at 5 000 and then doubling the length on each run until you get some useful results.

³Merge sort is another example of an $O(n \lg n)$ algorithm.

Real Jobs — Real People

Name: Wendy Roth
Title: President
Company: WRPD & Associates Inc.
Education: B.Math., University of Waterloo (1986)

Q: Please describe a bit about your career history.

A: I graduated from the University of Waterloo with a Bachelor of Mathematics, through the Co-Operative (Co-Op) program. The Co-Op program enabled me to work at a wide variety of IT functions and I very much enjoyed working in various environments and cultures. After graduation from Waterloo, I went directly into IT consulting and have been doing consulting ever since. I have been an external consultant for most of my career, but I have also held several internal consulting roles while working for organizations.

Roughly ten years ago, I decided to forge out on my own and begin consulting as a self-employed independent contractor. Being self-employed is never easy; it requires a great deal of dedication and discipline, but it is well worth the effort. I very much enjoy having my independence and selecting those assignments where I can provide the most value to organizations. What I like the most about my work is that I can come into an organization from the outside and make a difference to the way they operate, think, and manage.

My main areas of expertise are project management, change management, and business process around the use of technology. I also have expertise in the sales and marketing of IT. I have worked in a wide variety of industries, including sales and marketing, call centres, government, human resources, oil and gas, finance, and law.

Q: Which projects have you found really interesting and why?

A: I find projects that truly drive significant business change the most exciting and challenging. Change is never easy, especially business change. Organizations are too often set in their ways and not willing to move in different directions to meet evolving customer demands. IT is really just an enabler to business change, a tool to make business more effective, more efficient, and help better serve customers.

Q: What are some of the biggest challenges that organizations you have worked for are facing in today's environment? What are they doing about those challenges?

A: In today's economy, many organizations have great vision for technology solutions, but lack the capital investment. Justifying the cost of IT expenditures has always been a real challenge for many organizations. Then, once solutions are implemented, organizations have an even harder challenge measuring success. Every project should have clearly defined, measurable success factors, so an organization knows when it can say "job well done".

Also, the disciplines of project management and change management provide challenges to many organizations I have consulted with. I have served twenty-two clients over twenty years, and really only a small handful had the project and change management structures in place required to successfully implement technology solutions.

Q: What do you see as the "next big thing" in IT?

A: Predicting "next big things" is always a risky business, but I firmly believe that more emphasis will be placed on increasing business functionality in a wireless world — to be able to not just communicate with anyone, anytime, anywhere, but to be able to execute business transactions anytime and anywhere.

Q: What career advice do you have for someone completing a computing science degree today?

A: The strongest advice I would give to anyone considering a career, in *any* field, is to find something that you have a passion for and make it your career. I really don't have a job or career; I have a hobby that pays me well. If you find what you love to do and have a passion for it, you will naturally succeed. I firmly believe that I have been successfully self-employed for ten years because I believe in what I do, I have a passion for my work, and I strive for business excellence.

IT has so many facets (communication, programming, database design, project management, systems analysis), that you should try as many IT functions as you can before deciding which stream to specialize in. Experience as much as you can before focusing on one area.

Lastly, of course, you need to obtain as many academic credentials as possible. Education is one of the most critical foundations for a successful career. My degree from the University of Waterloo opened doors for me that would have not been opened otherwise. I very much enjoyed my time at Waterloo and firmly believe my studies helped pave the way for my success.